

TypeScript



Prof. David Fernandes de Oliveira
Instituto de Computação
UFAM

O que é TypeScript?

- TypeScript é um **superset** de JavaScript, isto é, faz tudo o que JavaScript faz e adiciona novos recursos à linguagem
 - Mantida pela Microsoft, em março de 2026 foi lançada a versão 6.0 da linguagem. Site oficial: <https://www.typescriptlang.org/>
- A principal razão para a adoção do TypeScript é a possibilidade de adicionar **tipagem estática** à linguagem JavaScript

```
// JavaScript  
let valor = "ufam";  
valor = 20; // valor mudou de string para um número, sem erros
```

```
// TypeScript  
let valor = "ufam";  
valor = 20; // ERRO - mudança de tipos não é permitida
```

TypeScript: JavaScript W x +

typescriptlang.org

Em Pt Search Docs

TypeScript

Download Docs Handbook Community Playground Tools

TypeScript is JavaScript with syntax for types.

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale.

Try TypeScript Now
Online or via npm

Editor Checks Auto-complete Interfaces JSX

```
const user = {
  firstName: "Angela",
  lastName: "Davis",
  role: "Professor",
}

console.log(user.name)
```

Property 'name' does not exist on type '{ firstName: string; lastName: string; role: string; }'.

TypeScript 6.0 is now available

What is TypeScript?

JavaScript and More

A Result You Can Trust

Sa

Esta página está disponível em Português

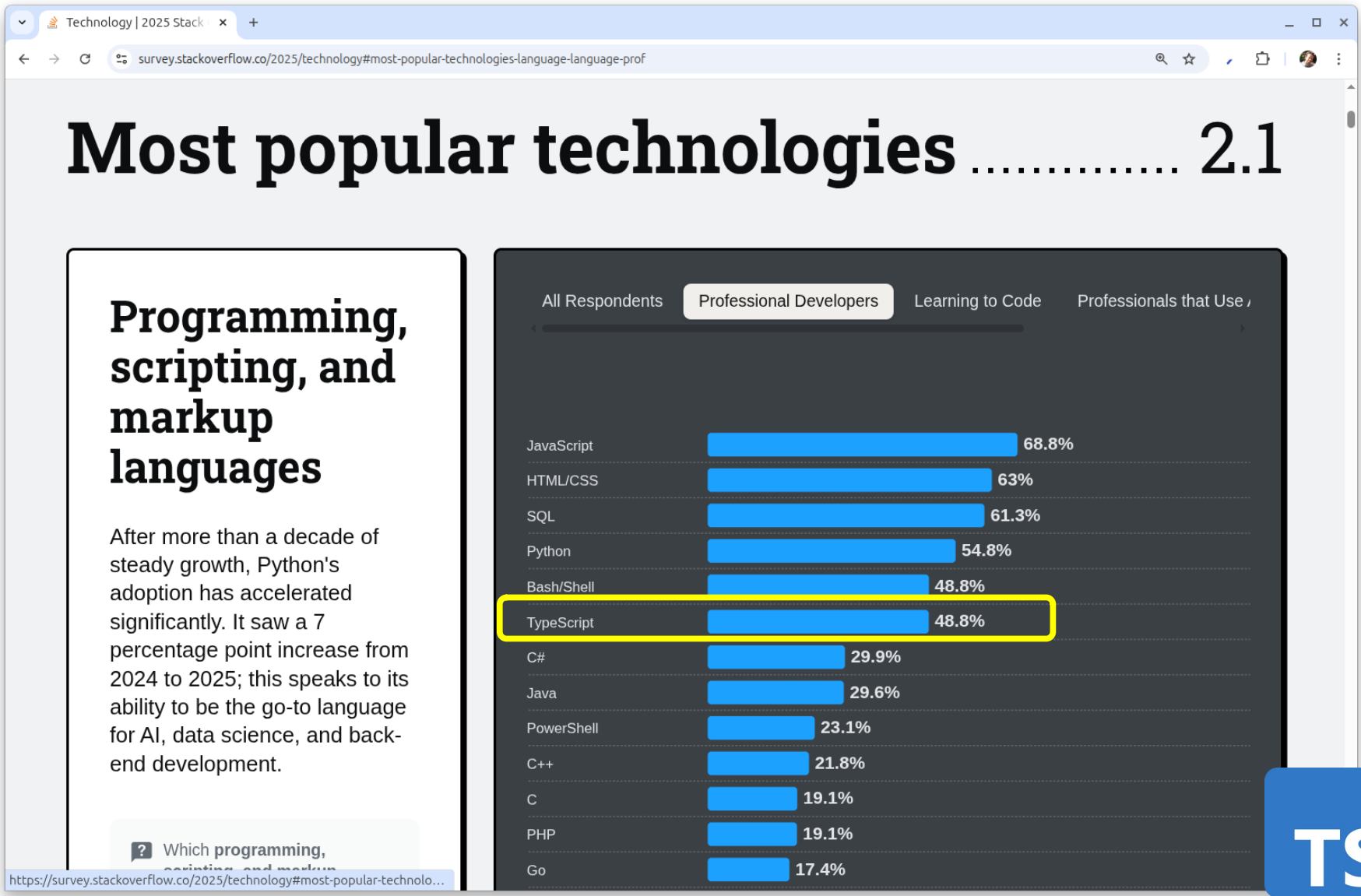
Trocar língua

Não perguntar novamente

TypeScript adds additional syntax to JavaScript. TypeScript code converts to JavaScript. TypeScript understands JavaScript and

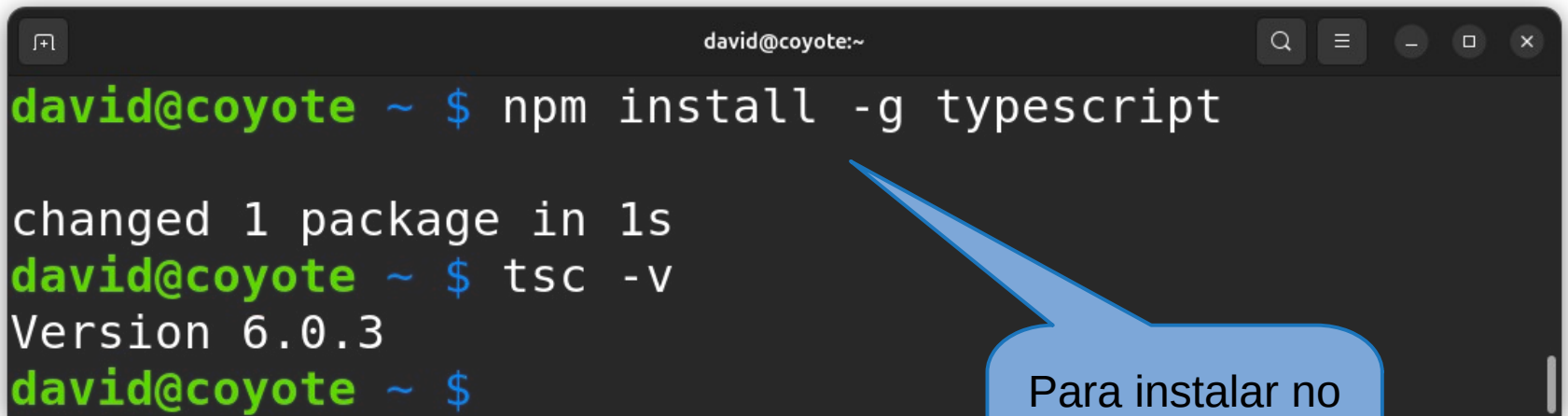


Stack Overflow Survey



Como instalar o TypeScript

- Para instalar o TypeScript, é preciso se certificar que o Node.js esteja instalado globalmente na máquina
- Com o NVM e Node.js instalado, é possível instalar o TypeScript globalmente usando o comando **npm install -g typescript**



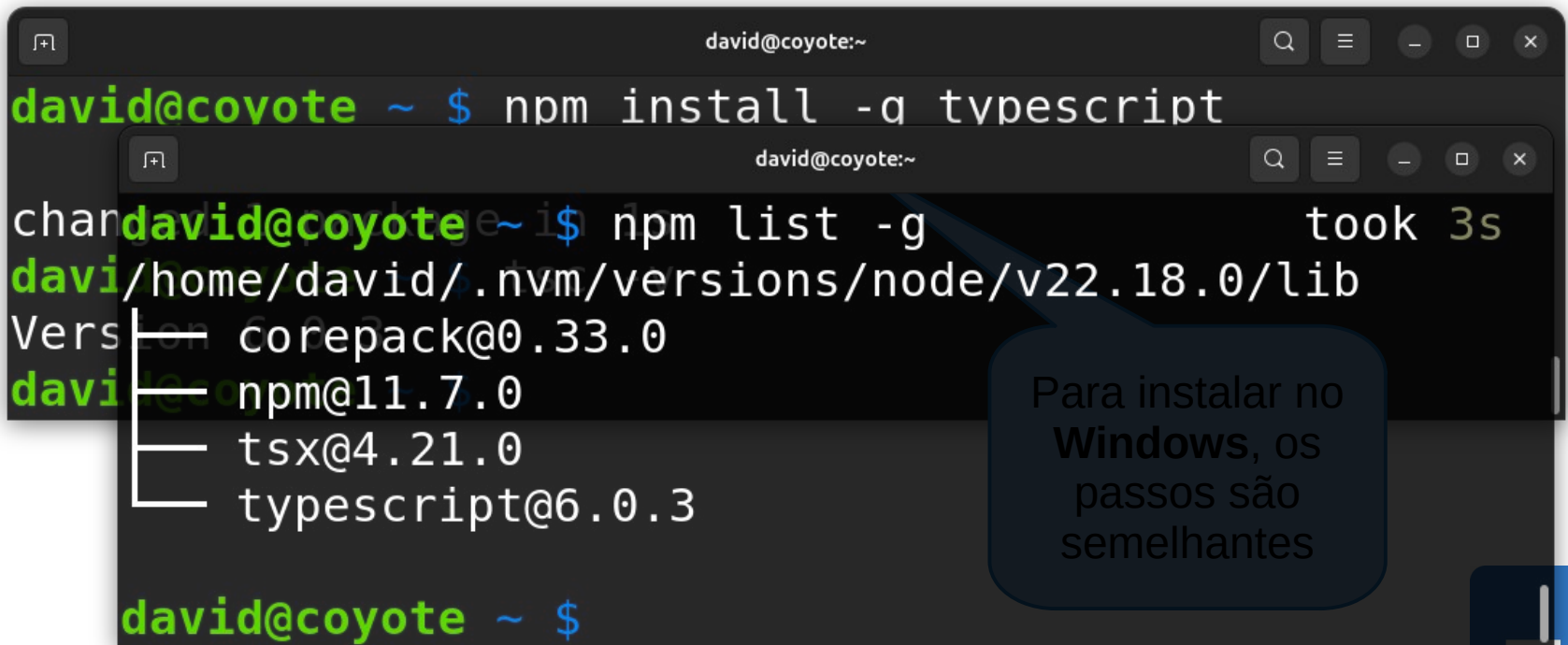
```
david@coyote:~$ npm install -g typescript
changed 1 package in 1s
david@coyote ~$ tsc -v
Version 6.0.3
david@coyote ~$
```

A terminal window with a dark background. The prompt is 'david@coyote:~'. The first command is 'npm install -g typescript', followed by the output 'changed 1 package in 1s'. The second command is 'tsc -v', followed by the output 'Version 6.0.3'. The prompt returns to 'david@coyote ~\$'.

Para instalar no **Windows**, os passos são semelhantes

Como instalar o TypeScript

- Para instalar o TypeScript, é preciso se certificar que o Node.js esteja instalado globalmente na máquina
- Com o NVM e Node.js instalado, é possível instalar o TypeScript globalmente usando o comando **npm install -g typescript**



The image shows a terminal window with the following commands and output:

```
david@coyote ~ $ npm install -g typescript
```

The command is executed, and the terminal shows the installation progress. A second terminal window is overlaid on top, showing the command to list global packages:

```
david@coyote ~ $ npm list -g
```

The output of the command is:

```
changed 1 package in 1s
david@coyote ~ $ npm list -g
/home/david/.nvm/versions/node/v22.18.0/lib
Version corepack@0.33.0
npm@11.7.0
tsx@4.21.0
typescript@6.0.3
```

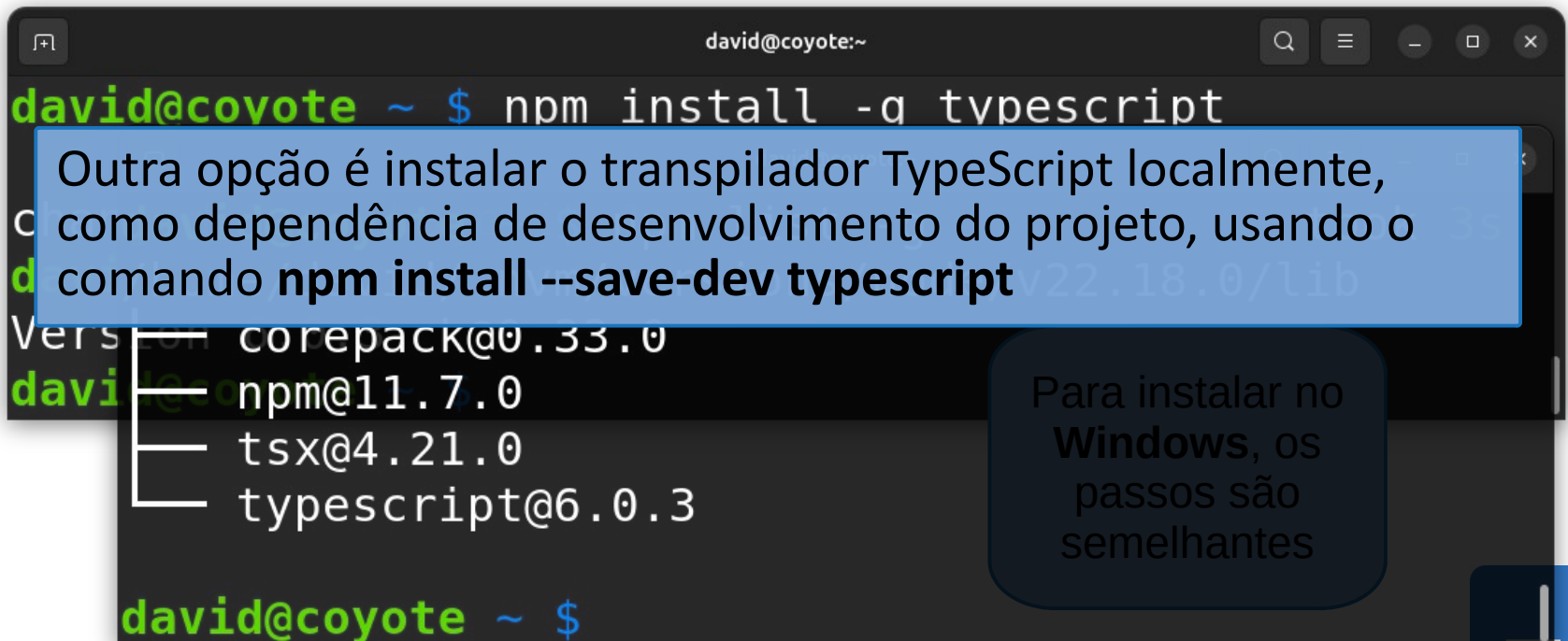
The terminal prompt is `david@coyote ~ $`.

Para instalar no
Windows, os
passos são
semelhantes



Como instalar o TypeScript

- Para instalar o TypeScript, é preciso se certificar que o Node.js esteja instalado globalmente na máquina
- Com o NVM e Node.js instalado, é possível instalar o TypeScript globalmente usando o comando **npm install -g typescript**



```
david@coyote:~$ npm install -g typescript
```

Outra opção é instalar o transpilador TypeScript localmente, como dependência de desenvolvimento do projeto, usando o comando **npm install --save-dev typescript**

```
Version: 0.33.0
david@coyote:~$ npm install --save-dev typescript
npm@11.7.0
  tsx@4.21.0
  typescript@6.0.3
```

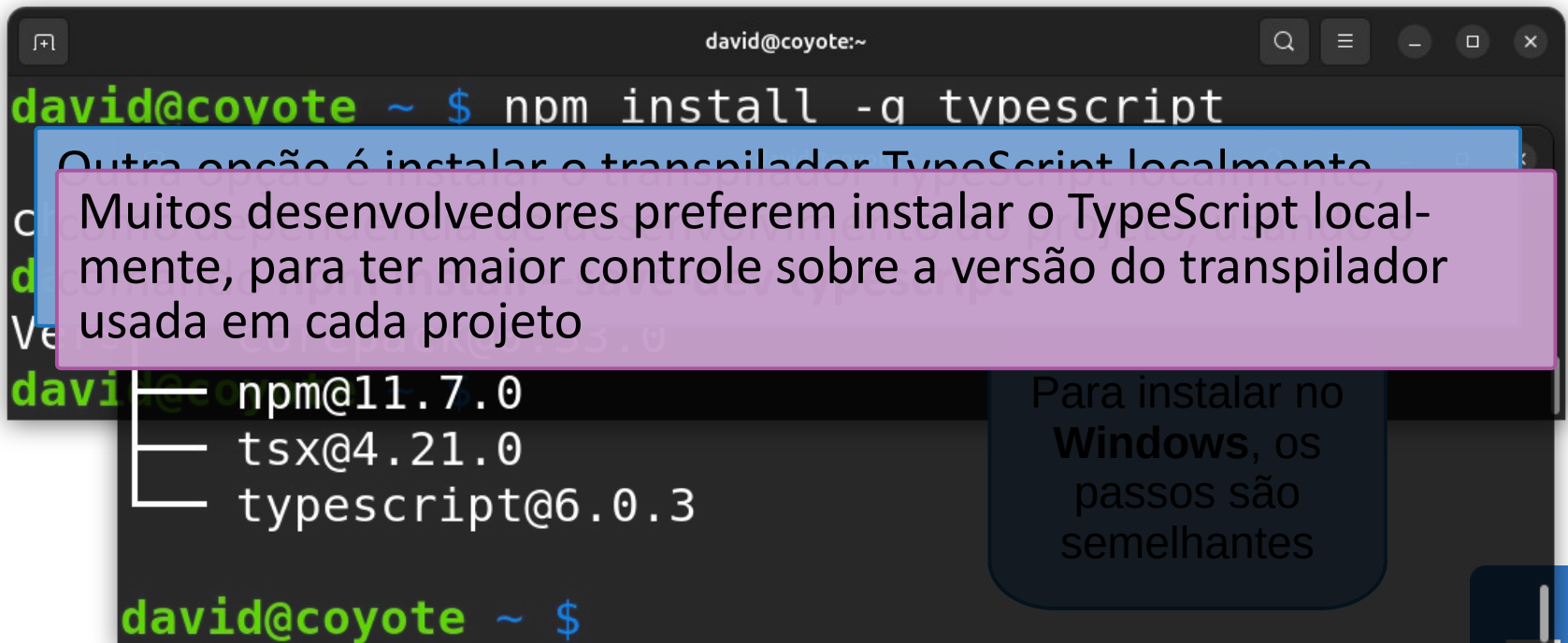
Para instalar no Windows, os passos são semelhantes

```
david@coyote ~ $
```



Como instalar o TypeScript

- Para instalar o TypeScript, é preciso se certificar que o Node.js esteja instalado globalmente na máquina
- Com o NVM e Node.js instalado, é possível instalar o TypeScript globalmente usando o comando **npm install -g typescript**



A terminal window with a dark background and light green text. The prompt is `david@coyote:~`. The command `npm install -g typescript` has been entered. Below the command, the output shows the installation of `npm@11.7.0`, `tsx@4.21.0`, and `typescript@6.0.3`. A semi-transparent purple box is overlaid on the terminal, containing the text: "Muitos desenvolvedores preferem instalar o TypeScript localmente, para ter maior controle sobre a versão do transpilador usada em cada projeto". Another semi-transparent dark blue box is overlaid on the right side of the terminal, containing the text: "Para instalar no Windows, os passos são semelhantes".

```
david@coyote ~ $ npm install -g typescript
Outra opção é instalar o transpilador TypeScript localmente
Muitos desenvolvedores preferem instalar o TypeScript local-
mente, para ter maior controle sobre a versão do transpilador
usada em cada projeto
david@coyote ~ $ npm@11.7.0
  tsx@4.21.0
  typescript@6.0.3
david@coyote ~ $
```

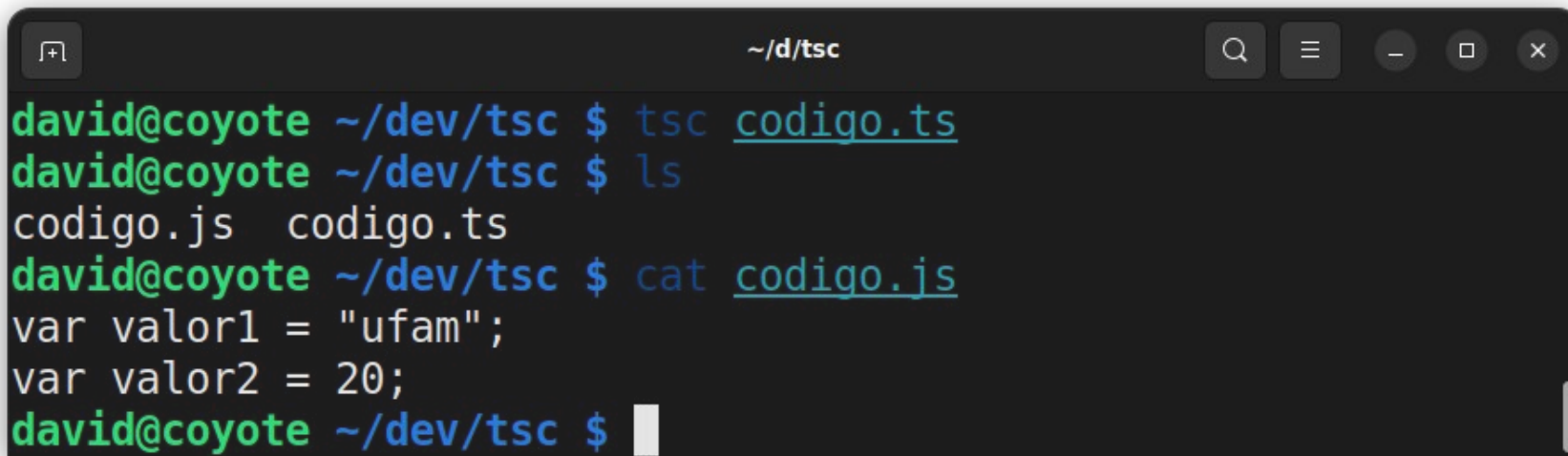


Como transpilar TypeScript

- Considere o código abaixo, salvo em um arquivo **codigo.ts**

```
let valor1 = "ufam";  
let valor2 = 20;
```

- Caso o TypeScript esteja instalado globalmente, o código acima pode ser **transpilado** em JavaScript usando **tsc**

A terminal window with a dark background and light-colored text. The window title is '~ / d / tsc'. The terminal shows a series of commands and their outputs. The user 'david@coyote' is in the directory '~/dev/tsc'. They run 'tsc codigo.ts', which produces 'codigo.js'. Then they run 'ls', which lists 'codigo.js' and 'codigo.ts'. Finally, they run 'cat codigo.js', which displays the transpiled JavaScript code: 'var valor1 = "ufam";' and 'var valor2 = 20;'.

```
~ / d / tsc  
david@coyote ~/dev/tsc $ tsc codigo.ts  
david@coyote ~/dev/tsc $ ls  
codigo.js  codigo.ts  
david@coyote ~/dev/tsc $ cat codigo.js  
var valor1 = "ufam";  
var valor2 = 20;  
david@coyote ~/dev/tsc $
```



Como transpilar TypeScript

- Considere o código abaixo, salvo em um arquivo **codigo.ts**

```
let valor1 = "ufam";  
let valor2 = 20;
```

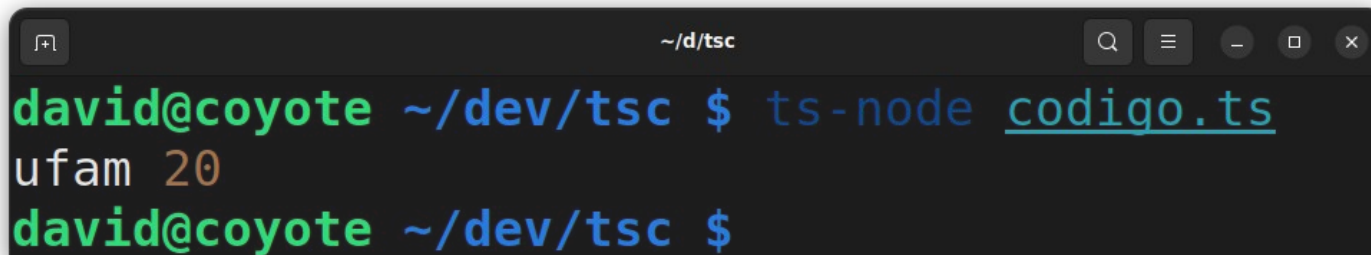
- Caso o TypeScript esteja instalado globalmente, o código
a Se o TypeScript estiver instalado localmente, pode-se utilizar o comando **npm tsc**.

```
~/.dev/tsc  
david@coyote ~/dev/tsc $ tsc codigo.ts  
david@coyote ~/dev/tsc $ ls  
codigo.js  codigo.ts  
david@coyote ~/dev/tsc $ cat codigo.js  
var valor1 = "ufam";  
var valor2 = 20;  
david@coyote ~/dev/tsc $
```

Rodando TypeScript com ts-node

- Código TypeScript pode ser rodado no Desktop sem necessidade de transpilá-lo para JS, usando o pacote **ts-node**
 - O ts-node pode ser instalado globalmente ou como dependência de desenvolvimento de um projeto

```
// Arquivo codigo.ts  
  
let valor1: string = "ufam";  
let valor2: number = 20;  
console.log(valor1, valor2)
```



A terminal window with a dark background. The title bar shows '~ / d / tsc'. The prompt is 'david@coyote ~/dev/tsc \$'. The command 'ts-node codigo.ts' is entered. The output is 'ufam 20'. The prompt changes to 'david@coyote ~/dev/tsc \$'.

```
~/d/tsc  
david@coyote ~/dev/tsc $ ts-node codigo.ts  
ufam 20  
david@coyote ~/dev/tsc $
```

- No entanto, esse recurso não está disponível para os browsers, que só podem rodar código javascript

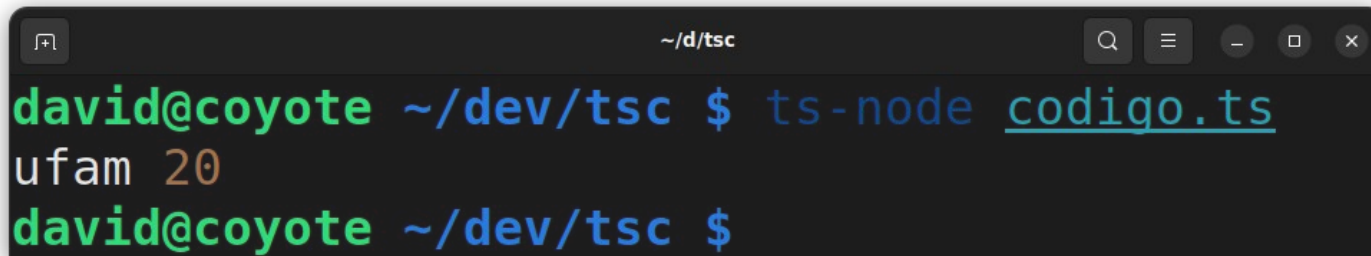


Rodando TypeScript com ts-node

- Código TypeScript pode ser rodado no Desktop sem necessidade de transpilá-lo para JS, usando o pacote **ts-node**
 - O ts-node pode ser instalado globalmente ou como dependência de desenvolvimento de um projeto

Além do **ts-node**, outro ambiente de execução Typescript que tem ganhado bastante popularidade é o **tsx**

```
let valor2: number = 20;  
console.log(valor1, valor2)
```

A terminal window with a dark background. The title bar shows '~ / d / tsc'. The prompt is 'david@coyote ~ / dev / tsc \$'. The command 'ts-node codigo.ts' is entered. The output is 'ufam 20'. The prompt changes to 'david@coyote ~ / dev / tsc \$'.

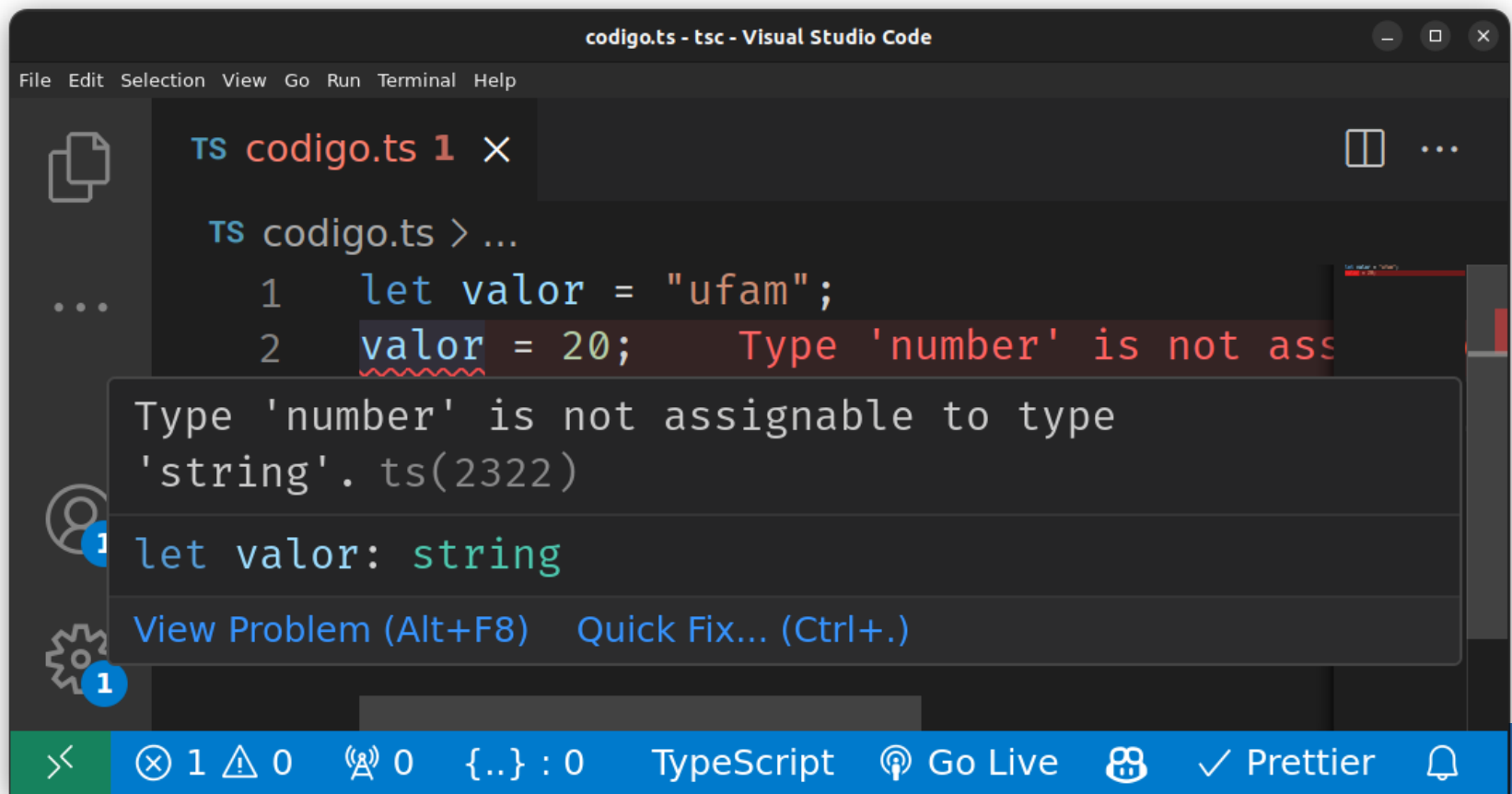
```
~/d/tsc  
david@coyote ~ / dev / tsc $ ts-node codigo.ts  
ufam 20  
david@coyote ~ / dev / tsc $
```

- No entanto, esse recurso não está disponível para os browsers, que só podem rodar código javascript



Intellisense do Visual Studio

- TypeScript **Intellisense** é útil para dar alertas de erros, auto-completes, dicas de digitação e popups de documentação



Intellisense do Visual Studio

- TypeScript **Intellisense** é útil para dar alertas de erros, auto-completes, dicas de digitação e popups de documentação



```
codigo.ts - tsc - Visual Studio Code
File Edit Selection View Go Run Terminal Help

~/d/tsc

david@coyote ~/dev/tsc $ tsc codigo.ts
codigo.ts:2:1 - error TS2322: Type 'number' is not assignable
to type 'string'.
    2 valor = 20;
      ~~~~~

Found 1 error in codigo.ts:2

david@coyote ~/dev/tsc $
```

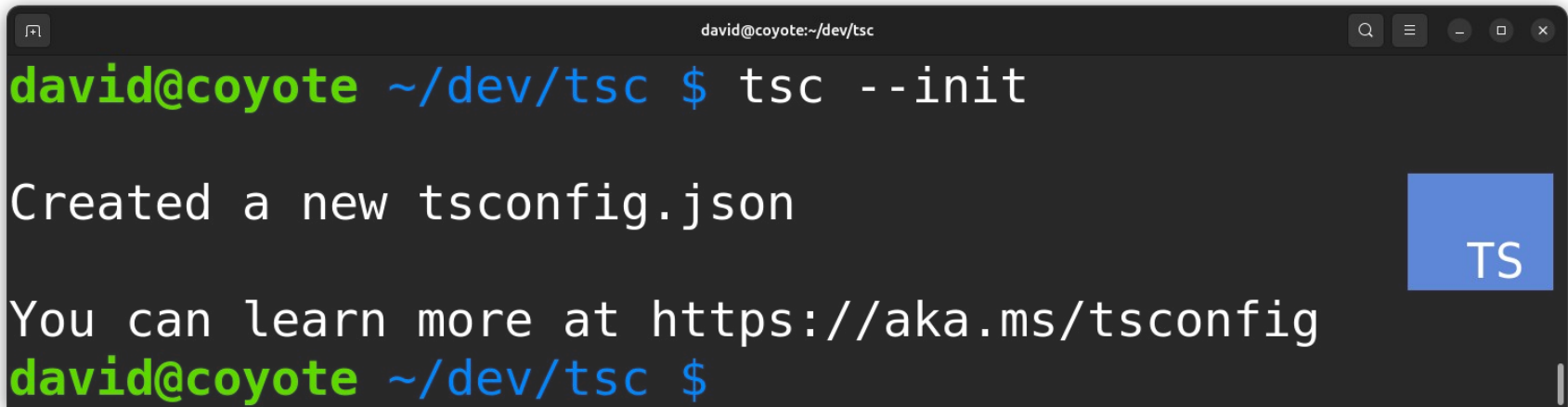
O mesmo erro
será mostrado
se tentarmos
transpilar
o código

TypeScript Go Live Prettier

TS

Configuração do TypeScript

- Para configurar as opções de transpilação do TypeScript, criamos um arquivo **tsconfig.json** no diretório raiz do projeto
- Para criar um arquivo tsconfig.json com um conjunto de opções pré-definidas, podemos usar o comando **tsc --init**



```
terminal
david@coyote:~/dev/tsc
david@coyote ~/dev/tsc $ tsc --init

Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
david@coyote ~/dev/tsc $
```

A terminal window with a dark background. The title bar shows 'david@coyote:~/dev/tsc'. The prompt is 'david@coyote ~/dev/tsc \$'. The command 'tsc --init' is entered. The output is 'Created a new tsconfig.json' followed by 'You can learn more at https://aka.ms/tsconfig'. The prompt is now 'david@coyote ~/dev/tsc \$'. A small blue square with 'TS' is in the bottom right corner of the terminal window.



Configuração do TypeScript

- Para configurar as opções de transpilacão do TypeScript, crie um arquivo `tsconfig.json` no raiz do projeto.
- Algumas opções importantes do **tsconfig.json**:
 - **target**: versão do JS para qual os arquivos serão compilados.
 - **module**: sintaxe usada para os módulos nos arquivos .js emitidos.
 - **outDir**: pasta de destino dos arquivos compilados.
 - **include**: vetor com os diretórios ou arquivos a serem compilados.
 - **exclude**: vetor com os diretórios ou arquivos devem ser ignorados pelo transpilador.

You can learn more at <https://aka.ms/tsconfig>

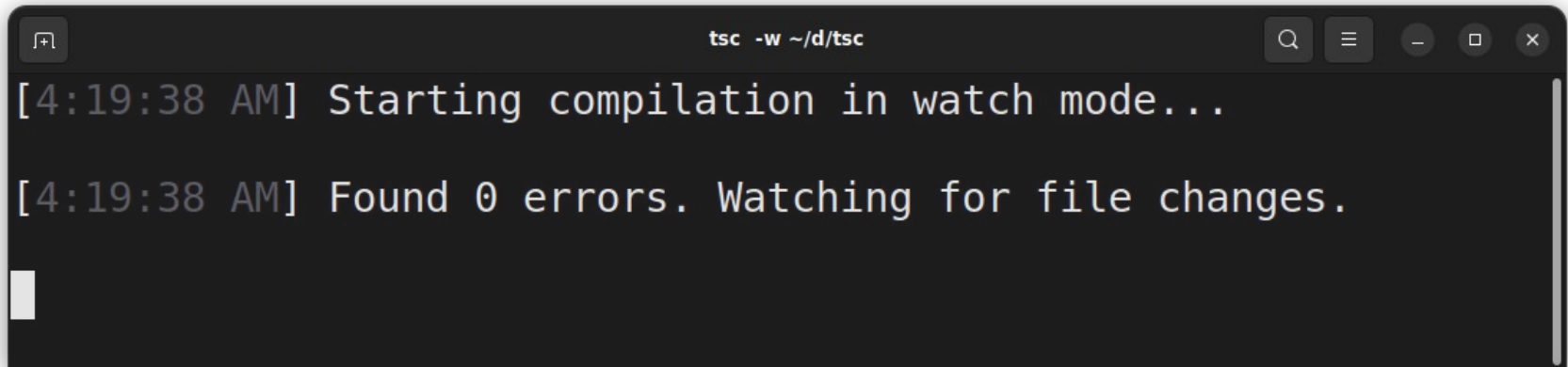
david@coyote ~/dev/tsc \$

TS

TS

Configuração do TypeScript

- Com o **tsconfig.json** criado, podemos usar o comando **tsc -w** ou **tsc --watch** para transpilar todos os arquivos TypeScript automaticamente quando algum código for editado

A terminal window with a dark background. The title bar at the top reads 'tsc -w ~/d/tsc'. The terminal output shows two lines: '[4:19:38 AM] Starting compilation in watch mode...' and '[4:19:38 AM] Found 0 errors. Watching for file changes.' The window has standard macOS window controls (zoom, search, menu, close) on the right side.

```
tsc -w ~/d/tsc
[4:19:38 AM] Starting compilation in watch mode...
[4:19:38 AM] Found 0 errors. Watching for file changes.
```



Tipos Primitivos

- A linguagem JavaScript possui 6 tipos de dados principais
 - string, number, bigint, boolean, undefined e null
- No TypeScript, podemos definir o tipo de uma variável adicionando **:<type>** após a declaração da variável

```
let id: number = 5;  
let primeiroNome: string = 'Danielle';  
let possuiCachorro: boolean = true;  
  
let valor: number; // Declaração sem atribuição  
valor = 5
```

Tipos Primitivos

- Em muitos casos a declaração de tipos primitivos não é obrigatória, pois o TypeScript pode inferir os tipos das variáveis

```
let id = 5; // TS agora sabe que é um número
let primeiroNome = 'Danielle'; // TS agora sabe que é string
let possuiCachorro = true; // TS agora sabe que é booleano
possuiCachorro = 'sim' // Erro
```

- Também podemos definir que a variável pode ter uma **união** de tipos, isto é, pode ter mais de um tipo

```
let idade: string | number;
idade = 26;
idade = '26';
```



Tipos Primitivos

- Em muitos casos a declaração de tipos é

The screenshot shows the Visual Studio Code editor with a file named `tipos_implicitos.ts`. The code contains several lines of TypeScript code. Line 1: `let id = 5; // TS agora sabe que é um número`. Line 2: `let possuiCachorro: boolean`. Line 3: `let possuiCachorro = true; // TS agora sabe que é booleano`. Line 4: (empty). Line 5: `possuiCachorro = "sim"; // Erro Type 'string' is not assignable to type 'boolean'`. Line 6: (empty). A blue callout bubble points to the `boolean` type in line 2, containing the text: "Intellisense do Visual Studio reconhece os tipos implícitos". The status bar at the bottom shows 1 error, 0 warnings, and 0 info messages. The file encoding is UTF-8, and the language is TypeScript.

```
TS tipos_implicitos.ts 1 x
TS tipos_implicitos.ts > ...
1 let id = 5; // TS agora sabe que é um número
2 let possuiCachorro: boolean // TS agora sabe que é string
3 let possuiCachorro = true; // TS agora sabe que é booleano
4
5 possuiCachorro = "sim"; // Erro Type 'string' is not assignable to type 'boolean'
6
```

```
idade = 26;
idade = '26';
```

Tipos literais e o tipo Any

- Além dos tipos gerais como **string** e **number**, podemos nos referir a valores específicos em posições de tipo

```
// Tipo de união com um tipo literal em cada posição  
let corFavorita: 'vermelho' | 'azul' | 'verde' | 'marrom';  
  
corFavorita = 'azul';  
corFavorita = 'amarelo'; // ERRO
```

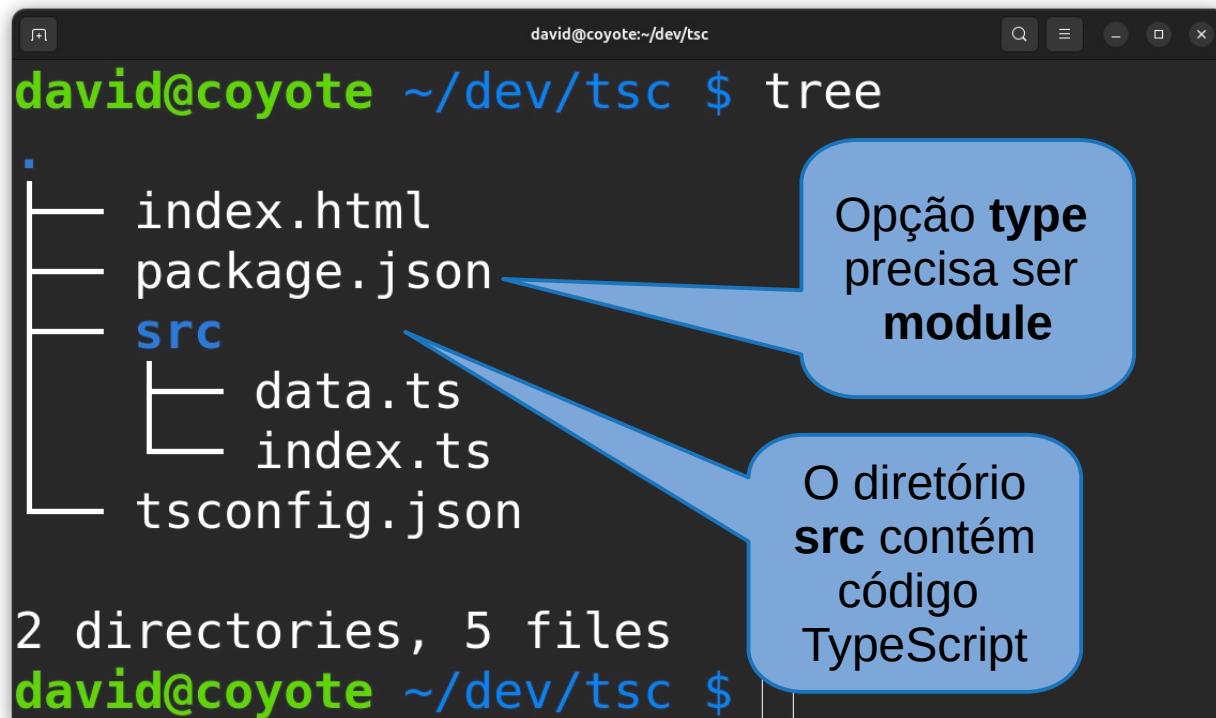
- O tipo **any** é um tipo genérico usado quando o tipo de uma variável é desconhecido ou ainda não foi definido

```
let valor: any  
valor = 'zero';  
valor = 1  
valor = false
```

O tipo any deve ser evitado, pois ele tira o propósito da tipagem estática do Typescript

Execução no Browser

- Os browsers não suportam código Typescript, então o código precisa ser transpilado para JS para ser executado no browser
- Para exemplificar esse processo, considere os arquivos abaixo



```
david@coyote ~/dev/tsc $ tree
.
├── index.html
├── package.json
├── src
│   ├── data.ts
│   └── index.ts
└── tsconfig.json

2 directories, 5 files
david@coyote ~/dev/tsc $
```

Opção **type** precisa ser **module**

O diretório **src** contém código TypeScript

Execução no Browser

- Em nosso exemplo, considere que os arquivos **src/script.ts** e **src/data.ts** possuem os seguintes conteúdos, respectivamente

```
// Arquivo src/script.ts  
import { nomeCurso } from "../data.js";  
  
const container = document.getElementById("container");  
if (container) container.innerHTML = nomeCurso;
```

```
// Arquivo src/data.ts  
export const nomeCurso: string = "Programação Web IComp/UFAM";
```

Note que o
arquivo **data.ts**
usa recurso
de tipagem do
TypeScript

Execução no Browser

- Através do arquivo **tsconfig.json** podemos definir um conjunto de opções de transpilação de código TypeScript para JavaScript

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./build",
    "module": "esnext",
    "target": "esnext",
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "strict": true,
    "verbatimModuleSyntax": true,
    "isolatedModules": true,
    "noUncheckedSideEffectImports": true,
    "moduleDetection": "force",
    "skipLibCheck": true
  },
  "include": ["src"]
}
```



Execução no Browser

- Como o código será transpilado na pasta **build**, podemos importar o arquivo **build/script.js** de dentro do **index.html**

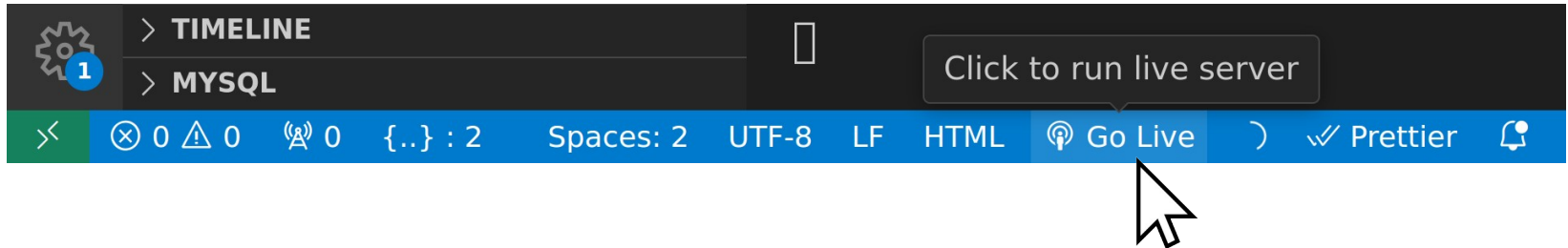
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Document</title>
  </head>
  <body>
    <div id="container"></div>
    <script type="module" src="../build/script.js"></script>
  </body>
</html>
```

Importante para
o uso de ES6
modules no
arquivo JS

Código
transpilado de
TypeScript
para JavaScript

Execução no Browser

- Para rodar o código, ative o **live server** do Visual Studio

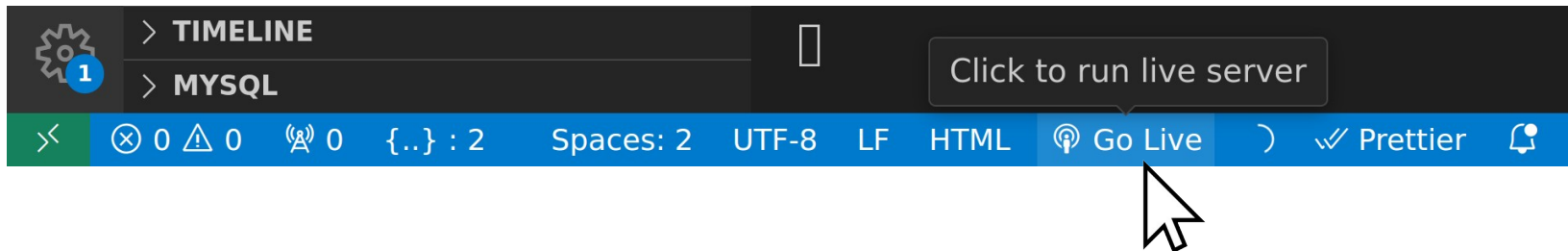


- A partir disso, o código poderá ser acessado pelo browser



Execução no Browser

- Para rodar o código, ative o **live server** do Visual Studio



- Alternativa: usar **bundlers** (como Vite, Parcel, Webpack). Essas ferramentas identificam todos os imports e transformam todo o código em um único arquivo .js compatível com o browser.

Programação Web IComp/UFAM

Vetores em TypeScript

- Em TypeScript, podemos definir os tipos de dados de um vetor:

```
// Vetor com apenas números
```

```
let vetor1: number[] = [1, 2, 3, 4, 5];
```

```
vetor1.push(6);
```

```
vetor1.push('7'); // Erro
```

```
// Vetor com apenas strings
```

```
let vetor2: string[] = ['Danielle', 'Marcos', 'Amanda'];
```

```
// Vetor com apenas objetos
```

```
let vetor3: object[] = [  
  {nome: 'O mundo assombrado por demônios', autor: 'Carl Sagan'},  
  {nome: 'Livro sobre nada', autor: 'Manoel de Barros'},  
];
```

```
// Vetor que pode conter qualquer tipo de dado
```

```
let vetor4: any[] = ['hello', 1, true];
```

Vetores em TypeScript

- Também podemos usar **tipos união** para definir vetores contendo diferentes tipos de dados:

```
let vetor: (string | number | boolean)[] = ['Danny', 1, true];  
vetor.push(100);
```



Tuplas

- O TypeScript possui um tipo chamado **Tupla**, que é um vetor com comprimento e tipos predefinidos para cada índice

```
let pessoa: [string, number, boolean];  
pessoa = ['Danielle', 1, true];
```

- As tuplas são muito úteis porque permitem que cada elemento do vetor seja de um tipo conhecido de valor

```
let funcionarios: [number, string, string, string?][];  
funcionarios = [  
  [0, "Danielle", "danielle@minhaempresa.com"],  
  [1, "João", "joão@minhaempresa.com"],  
  [2, "Maria", "maria@minhaempresa.com", "Supervisor"],  
];
```

atributo
opcional

Enum

- Enums são uma funcionalidade do TypeScript criada para facilitar o manuseio de grupos de **constantes nomeadas**
- Sua única finalidade é aumentar a **legibilidade** do código

```
enum StatusCodes {  
  NOT_FOUND = 404,  
  OK = 200,  
  ACCEPTED = 202,  
  BAD_REQUEST = 400  
}
```

```
console.log(StatusCodes.NOT_FOUND); // logs 404  
console.log(StatusCodes.SUCCESS); // logs 200
```

Ao invés de usar o valor 200 ao longo do código, podemos usar **StatusCodes.OK**

Por convenção, os nomes das constantes são em letras maiúsculas

Funções

- No TypeScript, podemos definir funções cujos argumentos e cujo retorno são tipados

```
function sum1(n1: number, n2: number): number {  
    return n1 + n2;  
}
```

- Se uma função não possui retorno, então podemos declarar que a função retorna **void**

```
function sum2(n1: number, n2: number): void {  
    console.log(n1 + n2);  
}
```

Funções

- Também é possível declarar alguns atributos como **opcionais**

```
function sum3(n1: number, n2: number, n3?: number): number {  
    return n1 + n2 + (n3 ?? 0)  
}
```

atributo
opcional

- Utilizando o **operador spread**, podemos criar funções com um número arbitrário de argumentos

```
function sum4(...numbers: number[]): number {  
    return numbers.reduce((acc, n) => acc + n, 0);  
}
```

Objetos

- Os objetos em TypeScript devem ter todas as propriedades e tipos de valores corretos

```
let person: {  
  nome: string;  
  endereco: string;  
  possuiPets: boolean;  
};  
  
person = {  
  nome: 'Danielle',  
  endereco: 'Rua André Araújo, 123',  
  possuiPets: true,  
};  
  
person.possuiPets = 'Sim'; // ERRO  
  
// ERRO: faltando a propriedade possuiPets  
person = { nome: 'João', endereco: 'Rua A, 234' };
```

Interfaces

- Para definir a assinatura de um objeto, podemos criar novos tipos de dados através das **interfaces**

```
interface Pessoa {  
  nome: string;  
  endereco: string;  
  possuiPets: boolean;  
}  
  
let pessoa1: Pessoa = {  
  nome: 'Danielle',  
  endereco: 'Rua André Araújo, 123',  
  possuiPets: true,  
};  
  
let pessoa2: Pessoa = {  
  nome: 'Marcelo',  
  endereco: 'Rua Maceió, 64',  
  possuiPossui: false,  
};
```

Interfaces

- Os métodos de objetos podem ser declarados através de assinaturas de funções
- Tais assinaturas podem ser criadas usando funções JavaScript comuns ou arrow functions

```
interface Cumprimentos {  
  digaOla(nome: string): string;  
  digaAdeus: (nome: string) => string;  
}
```

Classes em Typescript

- Podemos definir os tipos de cada propriedade em uma classe

```
class Pessoa {  
  nome: string;  
  possuiPets: boolean;  
  idade: number;  
  
  constructor(n: string, p: boolean, i: number) {  
    this.nome = n;  
    this.possuiPets = p;  
    this.idade = i;  
  }  
  
  digaOla() {  
    return `Olá, meu nome é ${this.nome}`;  
  }  
}  
  
const pessoa1 = new Pessoa('Danielle', false, 21);  
const pessoa2 = new Pessoa('Marcelo', 'sim', 25); // E
```



Classes em Typescript

- Uma vez que a classe seja definida, podemos criar um array de pessoas que inclua apenas objetos da classe Pessoa

```
const pessoa1 = new Pessoa('Danielle', false, 1);  
const pessoa2 = new Pessoa('Marcelo', true, 6);  
const pessoas: Pessoa[] = [pessoa1, pessoa2]
```

Note que o nome da classe pode ser usado como uma interface

Classes em Typescript

- Podemos adicionar modificadores de acesso, como **readonly**, **private**, **protected** e **public**, às propriedades de uma classe

```
class Pessoa {  
  readonly nome: string;  
  private possuiPets: boolean;  
  protected email: string;  
  public idade: number;  
  
  constructor(n: string, p: boolean, e: string, i: number) {  
    this.nome = n;  
    this.possuiPets = p;  
    this.email = e;  
    this.idade = i;  
  }  
  
  digaOla() {  
    return `Olá, meu nome é ${this.nome}`;  
  }  
}
```



Classes em TypeScript

Esta propriedade é imutável, só pode ser lida, nunca editada

- Podemos adicionar modificadores de acesso, como **readonly**, **private**, **protected** e **public**, às propriedades e métodos.

Apenas métodos da classe pode acessar ou modificar

```
class Pessoa {  
  readonly nome: string;  
  private possuiPets: boolean;  
  protected email: string;  
  public idade: number;  
  
  constructor(n: string, p: boolean, e: string, i: number) {  
    this.nome = n;  
    this.possuiPets = p;  
    this.email = e;  
    this.idade = i;  
  }  
  
  digaOla() {  
    return `Olá, meu nome é ${this.nome}`;  
  }  
}
```

Apenas métodos da classe ou subclasses podem acessar ou modificar

Pode ser acessada ou modificada de qualquer lugar

Classes em Typescript

- Podemos tornar nosso código mais conciso construindo propriedades de classe da forma abaixo

```
class Pessoa {  
  constructor(  
    readonly nome: string,  
    private possuiPets: boolean,  
    protected email: string,  
    public idade: number  
  ) {}  
  
  digaOla() {  
    return `Olá, meu nome é ${this.nome}`;  
  }  
}
```

Desta forma, as propriedades são atribuídas automaticamente no construtor

```
const pessoa = new Pessoa('Maria', false, 'maria@e.com', 21);  
console.log(pessoa.nome); // Maria
```

Classes em Typescript

- As classes também podem ser estendidas, assim como no JavaScript normal

```
class Programador extends Pessoa {  
  constructor(  
    readonly nome: string,  
    protected email: string,  
    public idade: number  
    public linguagens: string[]  
  ) {  
    // A chamada super chamada deve fornecer todos os  
    // parâmetros para classe base (Pessoa)  
    super(nome, false, email, idade);  
  }  
}
```

Aliases de tipo

- Além das interfaces, também é possível definir um tipo de objeto através de um **alias de tipo**

```
type Pessoa = {  
  nome: string;  
  idade: number;  
};  
  
function digaOla(pessoa: Pessoa) {  
  console.log(`Olá ${pessoa.nome}`);  
}  
  
digaOla({  
  nome: 'João',  
  idade: 48,  
}); // Olá João
```

Interfaces

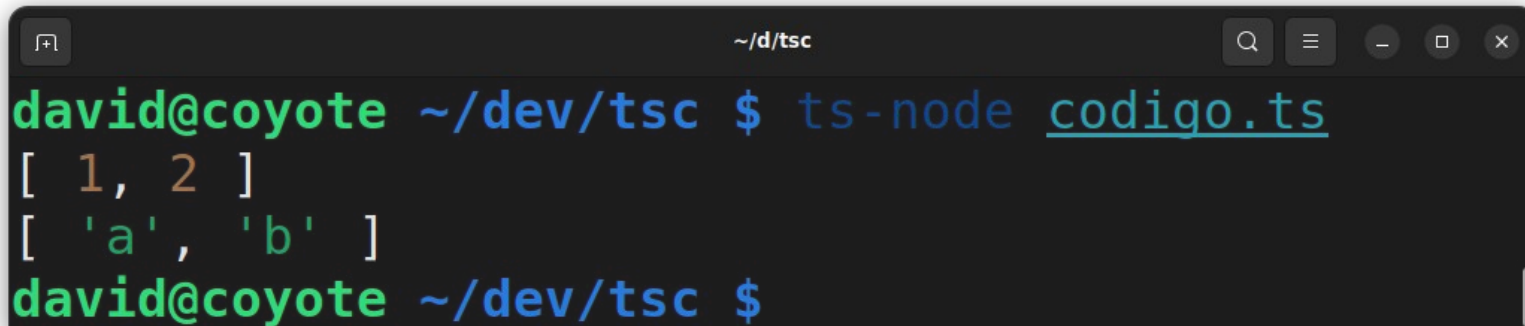
- As interfaces são muito semelhantes aos aliases de tipo e, em muitos casos, pode-se usar qualquer um deles
- A principal distinção é que os aliases de tipo não podem extendidos, enquanto que uma interface é sempre extensível

```
interface Animal {  
  name: string  
}  
  
interface Urso extends Animal {  
  gostaDeMel: boolean  
}  
  
const zeColmeia: Urso = {  
  name: "Zé Colmeia",  
  gostaDeMel: true,  
}
```

Genéricos

- Os generics permitem a definição funções (ou classes) que funcionam com qualquer tipo de dado

```
function createArray<T>(v1: T, v2: T): T[] {  
    return [v1, v2];  
}  
  
const arr = createArray<number>(1, 2);  
console.log(arr);  
  
const arr2 = createArray<string>("a", "b");  
console.log(arr2);
```



A terminal window with a dark background and light text. The title bar shows the path ~/d/tsc. The prompt is david@coyote ~/dev/tsc. The command ts-node codigo.ts is entered. The output shows two lines: [1, 2] and ['a', 'b']. The prompt then returns to david@coyote ~/dev/tsc \$.

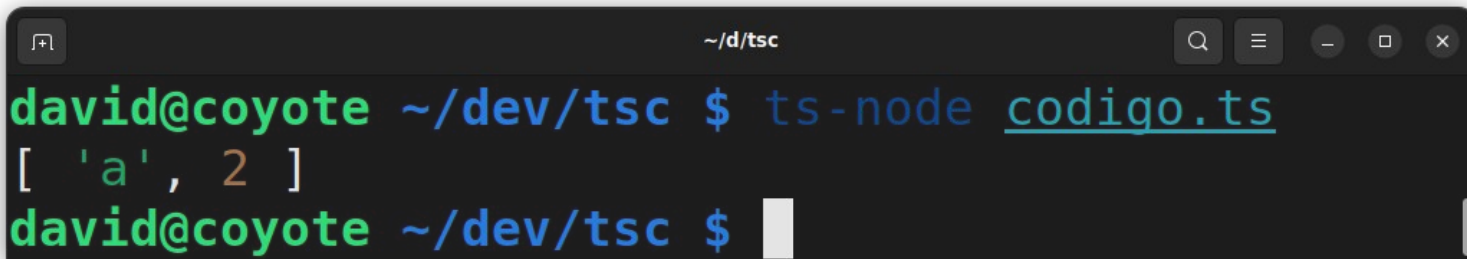
```
~/d/tsc  
david@coyote ~/dev/tsc $ ts-node codigo.ts  
[ 1, 2 ]  
[ 'a', 'b' ]  
david@coyote ~/dev/tsc $
```



Genéricos

- Também é possível deixar a função genérica para cada parâmetro, tal como a função createTupla abaixo

```
function createTupla<S, T>(value1: S, value2: T): [S, T] {  
  return [value1, value2];  
}  
  
const arr = createTupla<string, number>("a", 2);  
console.log(arr);
```



A terminal window with a dark background. The title bar shows '~ / d / tsc'. The prompt is 'david@coyote ~ / dev / tsc \$'. The command 'ts-node codigo.ts' is entered. The output is '['a', 2]'. The prompt is now 'david@coyote ~ / dev / tsc \$' with a cursor.

```
~ / d / tsc  
david@coyote ~ / dev / tsc $ ts-node codigo.ts  
[ 'a', 2 ]  
david@coyote ~ / dev / tsc $
```



Genéricos

- Genéricos podem ser usados para criar classes generalizadas

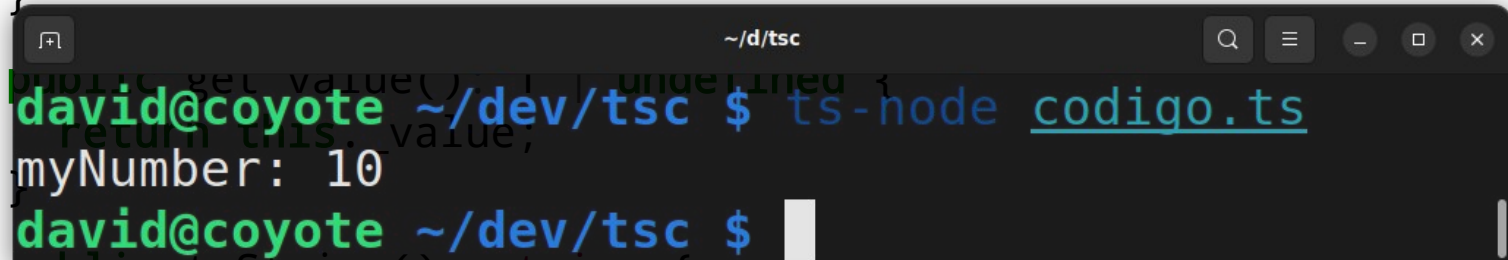
```
class NamedValue<T> {  
  private _value: T | undefined;  
  
  constructor(private name: string) {}  
  
  public set value(value: T) {  
    this._value = value;  
  }  
  
  public get value(): T | undefined {  
    return this._value;  
  }  
  
  public toString(): string {  
    return `${this.name}: ${this._value}`;  
  }  
}  
  
let value = new NamedValue<number>('myNumber');  
value.value = 10;  
console.log(value.toString());
```



Genéricos

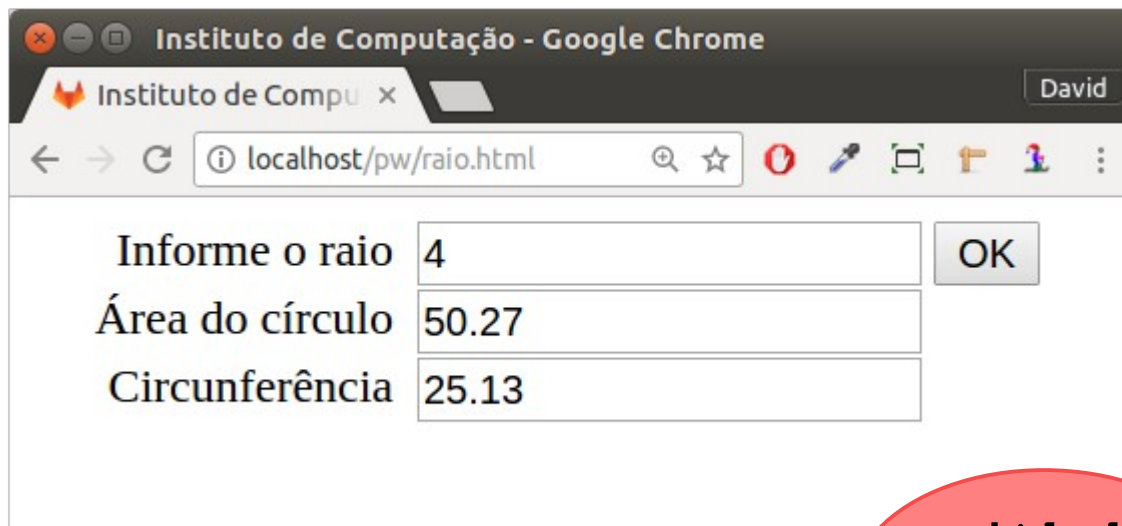
- Genéricos podem ser usados para criar classes generalizadas

```
class NamedValue<T> {  
  private _value: T | undefined;  
  
  constructor(private name: string) {}  
  
  public set value(value: T) {  
    this._value = value;  
  }  
  
  public toString(): string {  
    return `${this.name}: ${this._value}`;  
  }  
}  
  
let value = new NamedValue<number>('myNumber');  
value.value = 10;  
console.log(value.toString());
```



Exercício

- Usando TypeScript, escreva uma página similar à apresentada abaixo, que calcula a área e a circunferência de um círculo cujo raio é informado pelo usuário:



The screenshot shows a Google Chrome browser window titled "Instituto de Computação - Google Chrome". The address bar shows "localhost/pw/raio.html". The page content includes a form with the following elements:

Informe o raio	4	OK
Área do círculo	50.27	
Circunferência	25.13	

github
TS1

TS